

# MACE: A Multi-core Agentic Co-design Engine for Autonomous OpenPiton Bring-Up

Rana Umar Nadeem

CHIA A3 Hackathon Submission, September 2026

**Abstract**—MACE is an agentic AI system that builds and verifies multicore hardware on its own, running on the CHIA framework. Phase 1 adds `chia_openpiton`, a CHIA adapter exposing OpenPiton’s `configure`, `build`, `simulate`, `run`, and `collect` operations, validated with 208 tests. Phase 2 adds the MACE loop: a planner breaks a hardware objective into tasks, agents run them in parallel, and a deterministic integrator gates every change against Verilator simulation before accepting it. A failure-analysis agent diagnoses failures and the planner replans around them.

We tested MACE against two baselines, manual mesh scaling and a one-shot LLM prompt, on all three supported cores, with every result derived from a hardware run or RTL simulation, not an agent’s self-report. MACE added PicoRV32 as a third core, discovering and resolving several RTL bugs to reach the first passing Verilator run for that core, then drove it to a second, planner-directed pass on a compatible workload. It also passed 2x2 and 4x4 Ariane meshes where manual scaling stalled, resolving three further RTL/testbench bugs along the way. The one-shot baseline failed on all three cores in three different ways; MACE passed ariane and correctly stopped within its retry budget on the other two instead of reporting a false pass. The artifact is packaged with Nix, so setup is a single `nix develop`.

## 1. Introduction

Current agentic coding systems are typically benchmarked on isolated tasks or toy problems, which fails to capture the complexity of actual hardware construction. Bringing up a multicore system means navigating a large RTL codebase (OpenPiton spans hundreds of thousands of lines across network-on-chip, cache-coherence, tile, and platform logic), a fragile toolchain (version skew, cross-compilation bugs, checkout-specific issues), and verification that costs minutes to hours per attempt rather than milliseconds.

MACE is designed for this high-friction environment. Given a hardware objective, such as “bring up a 2x2 mesh” or “verify workload *W* passes,” MACE plans, implements, builds, simulates, diagnoses failures against Verilator output, and iterates. Every claim of success is gated by Verilator simulation; MACE never accepts an agent’s self-report.

### 1.1. Features

MACE contributes:

- `chia_openpiton` (§3): a CHIA adapter for OpenPiton with a full `configure/build/run/collect` interface, built the same way as CHIA’s `gem5` and `ESP` adapters, so it can be upstreamed directly.
- The MACE loop (§4): planning, parallel task execution, deterministic integration, and LLM-driven failure triage, tested end-to-end on hardware with cost, time, and task metrics.
- A third core, PicoRV32 (§5), brought up by fixing pre-existing bugs in its upstream OpenPiton integration, which OpenPiton’s continuous integration builds but never runs.
- An interactive CLI and two further capabilities (§6): Verilator code coverage as a second verification axis, and a second working LLM backend.

## 2. Background

CHIA provides the agent, tool, and dispatch layer this project builds on: colocated resource-pinned worker nodes, an LLM abstraction layer spanning multiple backends, and Ray-based remote dispatch and caching. OpenPiton is Princeton’s open-source, silicon-proven manycore research framework. Tiles connect through a 2D-mesh network-on-chip and speak a proprietary MESI-coherence protocol (L15, descended from OpenSPARC T1’s PCX/CPX bus) to reach the NoC. OpenPiton supports three tile cores: OpenSPARC T1 (`sparc`), Ariane/CVA6 (`ariane`, RV64GC, coherent), and PicoRV32 (`pico`, RV32I, non-coherent). Before this project, CHIA had no OpenPiton adapter.

## 3. Phase 1: The OpenPiton Adapter

**Design.** `OpenPitonWorkspaceNode` is a CHIA `ColocatedNode`. It exposes `configure`, `build`, `run`, `regress`, `collect`, and `clean` as resource-pinned remote functions. OpenPiton’s template preprocessor writes generated `.tmp.v` files directly into the source tree on every build, so concurrent builds cannot share a checkout. The adapter reserves one `openpiton` resource unit per checkout and gives each worker its own checkout instead of locking a shared one. Pure parsers read build and run verdicts directly from OpenPiton’s simulation transcripts (`sim.log`, `status.log`), using the same strings its RTL monitors emit. A verdict never comes from an agent’s claim.

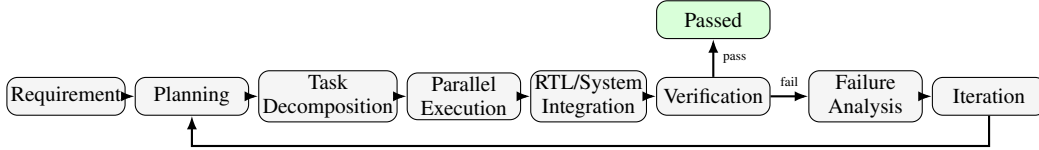


Figure 1. The MACE pipeline. Verification gates every integrated change against Verilator simulation rather than an agent’s self-report. The run ends when every gate workload passes; any failure triggers failure analysis and a replan rather than a bare retry.

**Environment hardening.** The build process surfaced four infrastructure bugs, fixed in one idempotent patch script: binutils 2.38+ splits `zicsr/zifencei` out of base RV64I and breaks the boot ROM’s assembly; GCC 15’s C23 default breaks a legacy two-argument function call; broken git symlinks on a Windows-mounted checkout turn into literal path-text files; and 66 files have CRLF-terminated shebang lines. These reflect the ordinary fragility of decades-old RTL codebases, which MACE navigates rather than avoids.

**Build-result caching.** `build()` skips the simulator entirely when a matching successful-build marker already exists on disk. A rebuild of an identical configuration measured 104s dropping to 0.00s: an agent iteration that only changes the test never rebuilds, with no extra loop-level code.

**Results.** 208 tests across three tiers (pure-unit, live-Ray with a stub simulator, and hardware) all pass. Hardware acceptance tests pass for a 1x1 Ariane build and run (`hello_world.c`, compiled and executed by the adapter itself) and for two independent checkouts building in parallel: 176s versus 324s serial. One acceptance test, 2x2 Ariane on a GCP worker, remains blocked by upstream infrastructure dependencies (§8).

## 4. Phase 2: The MACE Loop

**Pipeline.** Figure 1 shows the full loop. A planning agent turns a `MaceSpec` (core, target mesh, gate workloads, objective, budget) into a task DAG. Workers dispatch dependency-satisfied tasks in parallel through Ray, each pinned to its own checkout. A deterministic integrator, with no LLM in this step, applies accepted diffs to the canonical tree in dependency order, then runs every gate workload’s build and run. The gate is binary: every workload’s *verdict*, parsed from simulator output, must read `pass`. On any failure, a failure-analysis agent (restricted to read-only tools: `grep`, `status`, `collect`) diagnoses the transcript and proposes a fix. The planner replans around it.

**Gate workloads surface an RTL bug.** Writing the gate workloads (barrier-and-atomic-counter, producer/consumer, hart-id scatter/gather) surfaced a memory-visibility bug in this RTL/toolchain combination: a plain `volatile` read of a shared array element was not reliably visible from the same hart shortly after the write, while an atomic fetch-and-add-zero read of the same value always was. Every gate workload now reads shared values through this `atomic_read()` helper.

**Resource enforcement.** MACE checks all three `Budget` dimensions, iteration count, USD, and wall-clock, before each iteration starts, rather than only logging them afterward. Gate-workload checksums are verified at run start,

before any LLM call or checkout access, so a task cannot tamper with its own grading. Total spend across every run in this paper: \$37 of a \$300 GCP credit allocation, covering simulation compute and LLM inference; development itself ran on a \$20/month Claude subscription, outside that budget.

**Deterministic replay.** MACE cache-tags LLM, build, and run decisions and re-serves them at no extra token or compute cost. Reading CHIA’s cache and bypass implementation confirms this replays *decisions* (an LLM’s response text, a build or run’s pass/fail artifact), not the literal source edits an agent made through its tools; those remain git’s job.

## 5. A Third Core: PicoRV32

Integrating arbitrary core RTL into OpenPiton means navigating fundamental architectural differences rather than wiring up a standard bus. OpenPiton has no reusable core-to-NoC bridge: each core needs its own L15 adapter, and even Ariane’s AXI4 output goes unused, since a hand-written encoder taps CVA6’s internal pre-AXI cache-miss stream instead. The dividing line is whether the core has its own coherent cache, not whether it speaks a standard bus: a new core with one, as Ariane has, needs new coherence-adapter RTL, beyond this project’s timeline. PicoRV32 does not: a complete, upstream, TODO-free adapter (`pico_ll15_transducer.v` and friends) already exists in OpenPiton’s tile generation logic, unused, with no prior simulation coverage. OpenPiton’s CI builds it under Verilator, but its run job is commented out, targets a nonexistent stage, and even when live pointed at a different simulator.

**Toolchain reuse.** The RV32IMA/ILP32 multilib already present in our installed `riscv64-unknown-elf-gcc` covers PicoRV32 with no new toolchain install, confirmed with a trial assemble-and-link of the target diagnostic rather than by checking multilib metadata alone.

**Results.** The adapter extension (a new `Literal` entry, two `sims` flags, mirrored tests) landed with full unit coverage and no regressions to the existing `ariane` and `sparc` paths. The build passes in 57 seconds, faster than Ariane’s since pico’s build skips the bootrom/device-tree chain. The initial run reached verdict `maxcycles`.

The compiled binary’s symbol table and entry point matched the run’s configured trap addresses (`objdump -t/-f` against `symbol.tbl`), ruling out a bad build or toolchain mismatch. Its transcript matched a known-good passing-Ariane fixture line for line through the boot/reset/IOB-handshake trace; the fixture’s next line, `Hit Good trap`, never appeared for pico. This located the divergence to the point where the core must reach its trap address, not the boot code or the binary.

Waveform tracing found the root cause: three RTL/testbench bugs, PicoRV32’s `resethn/booted` self-boot gate, the manycore monitor’s `active_thread` tracking for pico’s tile, and a silicon BIST self-clear race that discarded pico’s first memory writes. PicoRV32 now **passes** (Simulation -> PASS (HIT GOOD TRAP)), its first passing Verilator run, found only because MACE tells “builds” apart from “runs correctly” instead of reporting a false pass.

**Multi-tile pico.** The fix generalizes to larger meshes. On 2x2 and 4x4 meshes, every tile independently reaches Hit Good trap on `addi.S` (Simulation -> PASS, cycle 5179750 at 2x2). `barrier_atomic.c` never runs on pico at any mesh size, for a simpler reason than first assumed: pico’s OpenPiton integration has no C compiler at all, only an assembler for `.S` sources, and no `crt.S/syscalls.c` exist for it. The diag never compiles, so every tile boots into an empty memory image and idles at its reset vector. This is a toolchain gap, not an RTL bug, and not the workload/mesh mismatch an earlier, unverified triage diagnosis assumed. Separately, pico’s AMO/atomic hardware path is directly verified: the upstream `amoadd_w.S` architecture test, pure assembly with no C dependency, passes on pico (Simulation -> PASS), confirming the atomic-memory-operation logic `barrier_atomic.c` would need is functionally correct at the instruction level.

## 6. Beyond the Proposal: Interface, Coverage, Portability

MACE adds three capabilities beyond the original proposal, each validated against a live backend or a full Verilator run.

The `mace` command starts an interactive shell modeled on Yosys and OpenROAD: `read_verilog`, `top_module`, `read_spec`, `set_core`, then `run`, giving MACE a standard EDA-style front end. If `top_module` names a core `chia_openpiton` does not support, `run` immediately flags the incompatibility instead of attempting a flawed integration.

Verilator code coverage (`run -coverage`) is a second verification axis beyond pass/fail. OpenPiton’s hand-written testbench never called Verilator’s coverage-write API, so a clean run produced no data until we patched it to call that API directly. A run (1x1 Ariane, `barrier_atomic.c`) reports 36.00% (8787/24311).

MACE’s loop also runs end to end, unmodified, against Google’s Vertex Gemini backend and OpenCode, confirmed with a full passing run. Vertex authenticates through Application Default Credentials, not a literal key, and we had to find which model was available (`gemini-2.0-*` returns 404; `gemini-2.5-flash` works).

## 7. Evaluation

We compare three approaches: (a) manual mesh scaling with a time log, (b) a single LLM prompt for a configuration

applied once with no tools, verification, or retry, and (c) the full MACE loop.

**Three cores, two baselines.** We ran (b) and (c) against the same objective, verify `barrier_atomic.c` on a 1x1 mesh, for all three supported cores, with the same LLM backend (Gemini 2.5 Flash) throughout (Figure 2, left).

(b) fails all three cores, each in a different way. On ariane, the one-shot config built but failed verification: a wrong guess. On sparc, it built and reached a run attempt that failed silently, with no verdict at all. On pico, `PitonConfig`’s construction-time validation rejected the LLM’s config (`l15_size=0`) before any hardware ran; (b) has no retry path for a malformed response.

(c) passes ariane: 4/4 tasks, 741.7s, about 4x (b)’s cost. A second ariane gate workload, `producer_consumer.c`, also passed (2118.02s, 2 tasks), coverage beyond one program. On sparc and pico, (c) exhausts its retry budget on `barrier_atomic.c` instead of passing; MACE’s triage and post-mortem told the two failures apart, and we verified both independently.

sparc’s failure has two layers. The first is an ISA gap: `barrier_atomic.c` calls `util.h`’s `ATOMIC_FETCH_OP` macro, literal RISC-V inline assembly. OpenSPARC T1 has no RISC-V instructions and no C diag environment at all, so no portable fix exists; writing new SPARC assembly is out of scope for an adapter project like MACE. Testing sparc with pure-assembly diagnostics instead, including `princeton-test-test.s`, OpenPiton’s upstream CI-verified sparc smoke test, avoids that gap entirely but surfaces a second, deeper one: every sparc run silently ran to `max_cycle` with no verdict at all, because `pc_cmp.v.pyv`’s `RTL_SPARC0` branch never assigned `active_thread`, unlike the `RTL_ARIANE0/RTL_PICO0` branches, which both assert it unconditionally once out of reset. Mirroring that pattern fixed the gap: the monitor now tracks all four sparc threads and reports `FAIL(TIMEOUT)` instead of silence. The threads themselves still never make forward progress, confirmed up to 100x the default timeout, an open finding distinct from the fixed observability bug.

pico’s original failure was first triaged as a workload/mesh mismatch (a 1x1 mesh cannot supply `barrier_atomic.c`’s multi-participant barrier), an unverified LLM diagnosis that turned out wrong: the cause, confirmed directly, is that `barrier_atomic.c` never compiles for pico at all (§5), independent of mesh size, not a return of the RTL bugs §5 already fixed. On `addi.S`, the single-hart workload §5 already verified by hand, the loop passes pico: `status=passed`, one task, 115.5s. That needed one new capability, a `CONFIG_RTL`: planner directive so a task can request `CONFIG_DISABLE_BIST_CLEAR` on top of the mesh’s defaults. The fix was named directly in the objective, so this shows the loop applying a known fix, not discovering one from scratch.

**The loop passes mesh shapes where manual scaling stalled.** Baseline (a) got stuck at 2x2 (build passes, run hangs) and never reached 4x4. The full loop passes both on every tile (Figure 2, right): 2x2 in 58.9s, 4x4 in 503.1s, each confirmed tile by tile. Getting there required fixing three

| Core   | (b) one-shot LLM                 | (c) full loop                    |
|--------|----------------------------------|----------------------------------|
| ariane | <b>fail</b> , 186.5s             | <b>pass</b> , 741.7s             |
| sparc  | <b>fail</b> , 766.0s             | <b>budget_exceeded</b> , 966.3s  |
| pico   | <b>fail</b> (unparseable), 32.6s | <b>budget_exceeded</b> , 1250.5s |

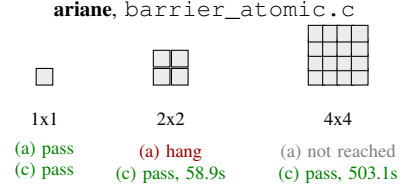


Figure 2. **Left:** all three cores, same 1x1 objective. (b) never passes. On sparc/pico, (c) exhausts its retry budget instead of reporting a false pass. Only ariane reaches a hardware **pass** under either baseline (logs: runs/baseline\_{b,c}\_output.log for ariane, runs/bench\_{sparc,pico}\_{b,c}.log for sparc/pico). **Right:** manual mesh scaling (a) vs. (c), ariane only. **Green**=pass, **red**=hang, **gray**=not reached. The loop passes both shapes (a) got stuck on.

latent bugs in OpenPiton’s shared code. A hardcoded trace-file name in `cva6.sv` made every tile write the same file, so tiles beyond 0 looked like they never booted. The upstream exit barrier in `syscalls.c`, used by every ariane diagnostic, polls a cross-tile atomic counter with a plain load, the same staleness bug `atomic_read()` already works around in our gate workloads. And the monitor’s `finish_mask` register was a fixed 32-bit Verilog `integer` under Verilator, capping verifiable coverage at 8 tiles: a 4x4 run reported **PASS** after only 8 of 16 tiles finished, as each tile’s execution trace showed. All three are fixed and verified with a full, tile-by-tile rerun.

Phase 2’s fan-out (two tasks, two checkouts, through baseline (c)’s path) measured 30s parallel versus 57s serial, matching Phase 1’s result.

One validation limitation remains: we proved the detect-failure, diagnose, replan, pass cycle only at the mechanism level (a stateful stub that fails once, then passes); it never fired across the passing runs above. This includes a forced extreme first config (a 128-byte, direct-mapped L1, confirmed in the build) that we and the planner both expected to fail. It passed instead, indicating the RTL’s coherence is robust to cache-capacity extremes rather than a failure of the recovery loop’s mechanism.

## 8. Limitations and Open Issues

**A dispatch configuration issue, fixed.** A task needing a tailnet-relayed GCP worker never ran, although that worker’s registration and heartbeat to the cluster worked throughout. CHIA sets three proxy environment variables on each node during `chia up`; a job submitted through `chia job submit` inherits them, but a manually launched driver, which every run here uses, does not unless the launching shell sets them. Exporting those values from the head’s live raylet took the task from a six-minute hang to a two-second run.

A later, independent dispatch attempt on the same topology reproduced the stuck symptom, with that fix already confirmed in place, ruling out a repeat of the same cause. Both tailnet relays’ logs and a raylet-level `strace` show zero new outbound activity, pointing at Ray’s scheduling under a proxied topology rather than CHIA. We filed this as a follow-up; it remains open.

**4x4 memory pressure, two causes, both fixed.** The one upstream-validated 4x4 shape first ran out of memory (22.33GB of 23.47GB, reported by Ray) under unconstrained parallel compilation. Limiting parallelism through

`MAKEFLAGS` was the obvious fix, but OpenPiton’s Verilator build step passed a bare, unlimited `-j` on its command line that silently overrode it, which we confirmed through `/proc`. Removing that `-j` lets `MAKEFLAGS` control parallelism. This exposed a smaller issue, a race in the boot ROM’s Makefile where one parallel job’s cleanup deletes a device-tree blob before another job reads it; serial compilation avoids it, so we build 4x4 with `MAKEFLAGS=-j1`.

**Future work: generality over one-off adapters.** All three cores here already had an OpenPiton/L15 adapter upstream: SPARC’s CCX transducer, Ariane’s L15 adapter inside CVA6, and PicoRV32’s transducer. A core without one, such as Ibex or Rocket Chip, needs that adapter written first. The existing three share a shape: a per-core RTL directory, a transducer between the core’s memory interface and L15, and a core-select arm in the tile template. Turning that shape into a template and using it to add 2-4 more cores is the direct next step, with MACE’s loop assisting a hardware engineer by verifying each iteration of the adapter RTL in Verilator, rather than generating adapters on its own. A second direction is a dashboard over the run database, showing dispatch, build, and verification as they happen instead of through raw trace files.

## 9. Conclusion

MACE shows that an agentic loop can plan, build, and verify multicore hardware, gated by RTL simulation instead of an agent’s word. Along the way, the project fixed environment and RTL bugs, added a third core, and passed 2x2 and 4x4 Ariane meshes where manual scaling stalled. Precision, not a single pass, is the point.